

CAD-ANALYZER

Fichas de Extração — LAJES

■ Esta ficha responde: dado um texto/polígono do DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_LAJE · RE_LAJE_H · caminhos A e B
2	Contorno e Área	LWPOLYLINE · fórmula Shoelace + diagrama anatômico
3	Espessura h=	extrair h= · range 7–40cm · casos especiais
4	Laje Sintética	clusters h= + diagrama · CLUSTER_RADIUS=500mm
5	Schema JSON Completo	FichaFase3Laje · todos os campos
6	Layers Canônicos	EST-LAJE-TEXT·Painéis·Vazio·encoding CP1252
7	Confidence	fórmula + diagrama · thresholds
8	Aberturas e Recortes	layer Vazio · is_void_layer() · Pilares
9	Exemplos Reais	L5 (1.0) · synth_0 (0.50) · L3 abertura
10	Pipeline Completo	fluxo E2E · checklist 10 passos

```
# Caminho A – ID explícito
RE_LAJE = re.compile(
    r'^(L\d+[A-Za-z]?|Y\d+[A-Za-z]?|X\d+[A-Za-z]?|
    r'|LAJ[-_]?d+|LAJE[-_s]*d+)$',
    re.IGNORECASE
)

# Caminho B – espessura h= (sem ID explícito)
RE_LAJE_H = re.compile(r'h\s*[=:]\s*([\d,.]+)', re.IGNORECASE)
# Ex: 'h=12' 'h = 14' 'h:10' → espessura em cm
```

Texto DXF	RE_LAJE?	RE_LAJE_H?	Resultado
L5	✓	—	codigo="L5"
L12A	✓	—	codigo="L12A"
Y1	✓	—	codigo="Y1"
LAJE 1	✓	—	codigo="LAJE 1"
h=12	—	✓	espessura=12.0cm
L5 h=12	✓	✓	codigo="L5", espessura=12.0cm
L	✗	—	sem número — não casa

Anatomia da Laje no DXF — Todos os Elementos

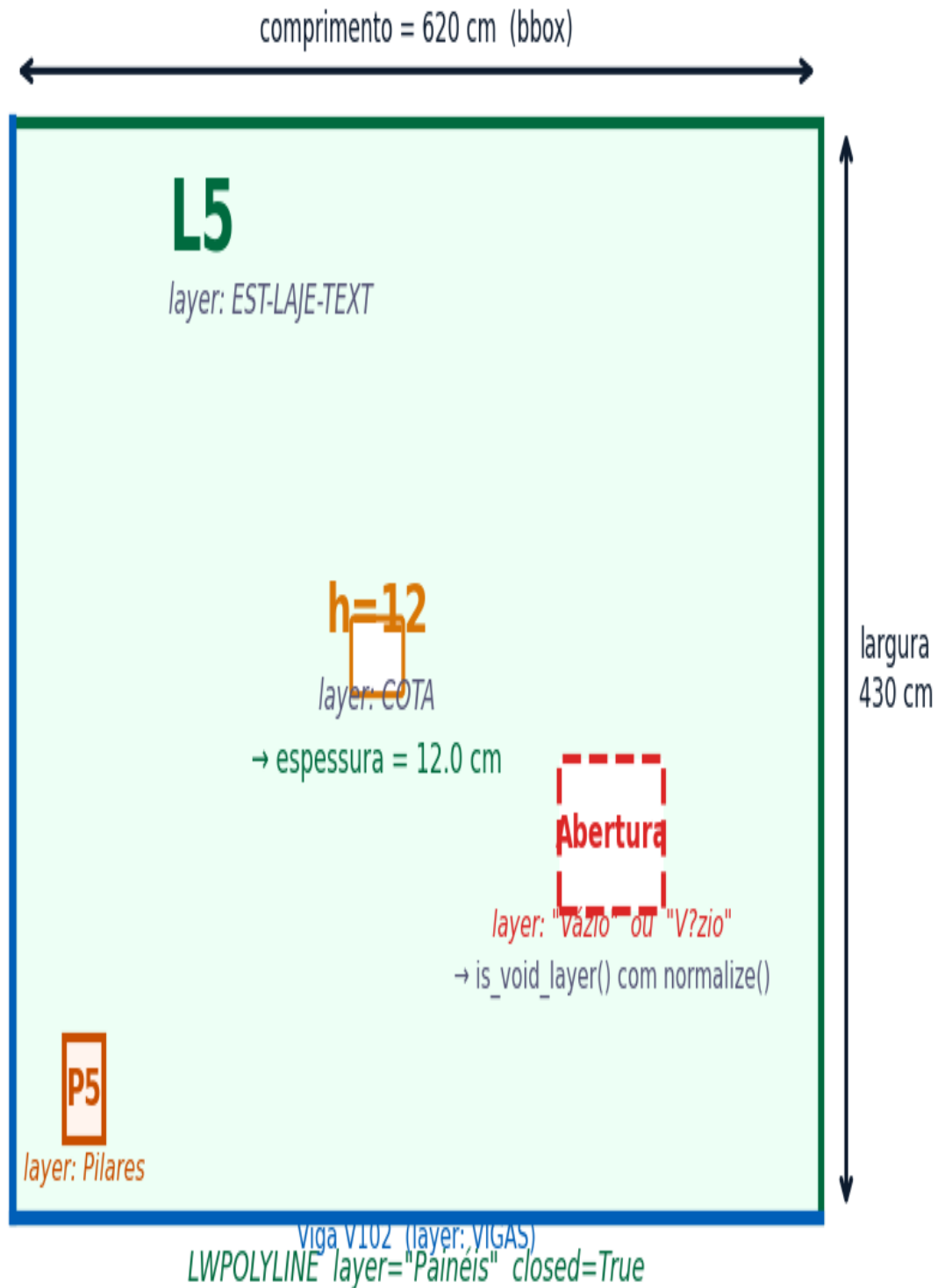


Figura 1 — Anatomia completa da laje: contorno, ID, h=, abertura, recorte pilar, vigas

```
LAJE_SEARCH_RADIUS = 1500.0 # mm

for e in msp.query('LWPOLYLINE'):
    pts = [(float(p[0]),float(p[1])) for p in e.get_points('xy')]
    is_closed = (getattr(e.dxf,'flags',0) & 1 == 1) or e.is_closed
    if is_closed and len(pts) >= 3:
        area = Polygon(pts).area
        if area > 50_000: # mm2 → provável LAJE
            laje_polys.append({'pts':pts,'layer':e.dxf.layer})
        elif area < 5_000: # mm2 → provável PILAR
            pilar_polys.append({'pts':pts,'layer':e.dxf.layer})

# Fórmula Shoelace (área sem shapely):
def area_shoelace(pts):
    n=len(pts); area=0.0
    for i in range(n):
        j=(i+1)%n
        area += pts[i][0]*pts[j][1] - pts[j][0]*pts[i][1]
    return abs(area)/2.0
```

```
def extrair_espessura(textos_h, laje_pos, raio=1500.0):
    lx, ly = laje_pos; candidatos = []
    for t in textos_h:
        m = RE_LAJE_H.search(t['text'])
        if not m: continue
        dist = math.hypot(t['x']-lx, t['y']-ly)
        if dist <= raio:
            val = float(m.group(1).replace(',', '.'))
            candidatos.append((val, dist))
    if candidatos:
        candidatos.sort(key=lambda x: x[1]) # mais próximo
        return candidatos[0][0]
    return 0.0
```

Valor	Válido?	Ação
h=12	SIM (7–40cm)	espessura=12.0
h=5	NÃO (< 7cm)	confidence -= 0.30, log "espessura inválida (norma)"
h=45	NÃO (> 40cm)	confidence -= 0.30, log "espessura fora do range"
ausente	—	espessura=0.0, confidence -= 0.30
2 valores conflitantes	—	usar o mais próximo ao centróide da laje

Laje Sintética — Geração por Clusters de $h=$

Entrada: textos $h=$ sem ID L^*

Saída: Lajes Sintéticas geradas

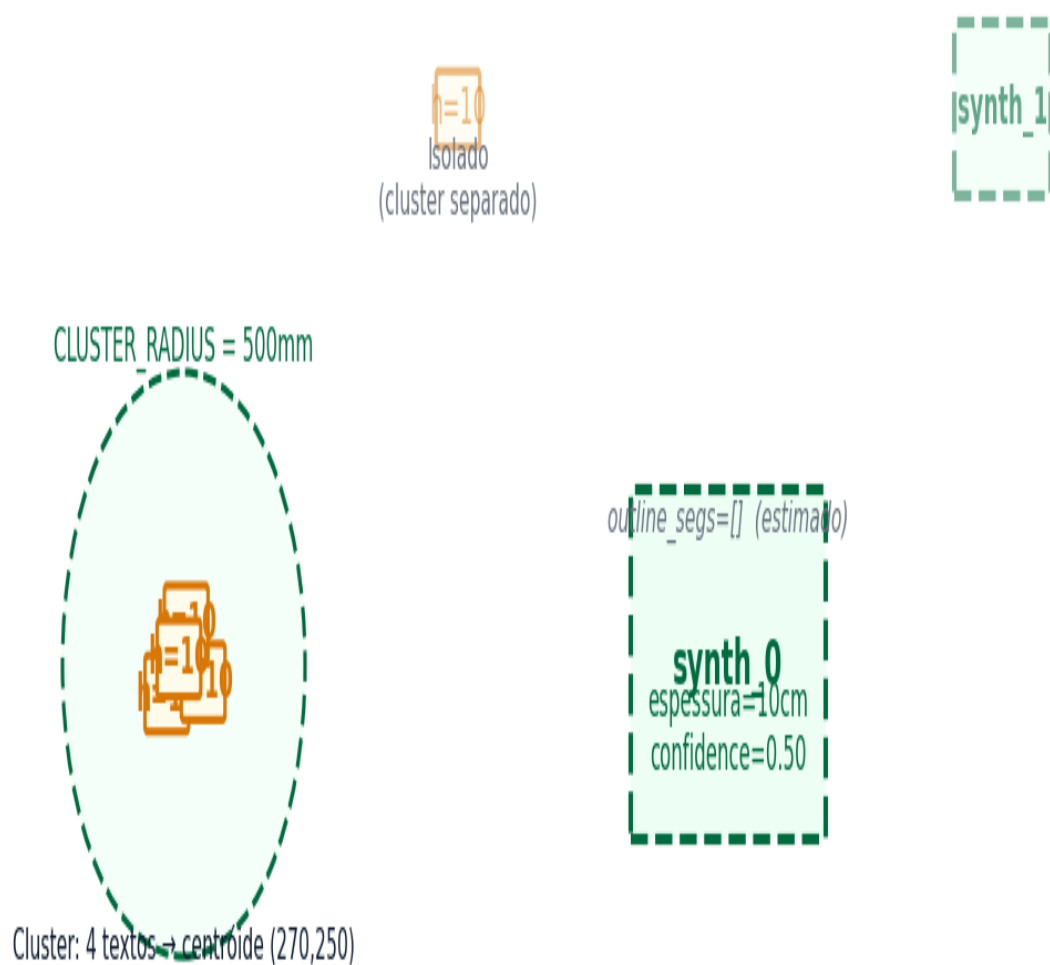


Figura 2 — Múltiplos textos $h=$ dentro de $CLUSTER_RADIUS$ → uma laje SYNTHETIC por cluster

```
CLUSTER_RADIUS = 500.0 # mm

def gerar_lajes_sinteticas(laje_dims):
    used=set(); clusters=[]
    for i,d in enumerate(laje_dims):
        if i in used: continue
        cluster=[d]; used.add(i)
        for j,d2 in enumerate(laje_dims):
            if j in used: continue
            if math.hypot(d['x']-d2['x'],d['y']-d2['y']) < CLUSTER_RADIUS:
                cluster.append(d2); used.add(j)
        clusters.append(cluster)
    result=[]
    for idx,cluster in enumerate(clusters):
        cx=sum(d['x'] for d in cluster)/len(cluster)
        cy=sum(d['y'] for d in cluster)/len(cluster)
        result.append({'id':f'synth_{idx}','name':'SYNTHETIC',
            'x':cx,'y':cy,'espessura':cluster[0]['h_val'],
            'confidence':0.50})
    return result
```

```
{
  "codigo": "L5", # str – ID ("synth_0" se sintética)
  "pavimento": "1_PAVIMENTO", "obra_nome": "ALIMONTI",

  "tipo": "macica", # "macica"|"pre_moldada"|"steel_deck"
  "espessura": 12.0, # float cm – de h=NN

  "dimensoes": {
    "comprimento": 620.0, # cm – bbox do contorno
    "largura": 430.0, # cm – bbox do contorno
    "espessura": 12.0
  },

  "outline_segs": [ # vértices em mm
    {"x":15000.0,"y":10000.0}, {"x":21200.0,"y":10000.0},
    {"x":21200.0,"y":14300.0}, {"x":15000.0,"y":14300.0}
  ],

  "aberturas": [{"pontos":[[5800,5900],[5900,5900],[5900,6100],[5800,6100]],
    "area":20000.0}],

  "vigas_around": ["V101","V102","V103"],
  "pilares_around": ["P5","P6","P7","P8"],

  "nivel": 2.80, "confidence": 0.70, "revisado": false
}
```


✗ ENCODING CRÍTICO: "Vázio" pode chegar como "V?zio" em DXF CP1252. Sempre use `is_void_layer()`.

Canônico	Aliases reais	Uso
SLAB_TEXT	EST-LAJE-TEXT, NOMENCLATURA, EST-TEXT	IDs de lajes (L1, L2...)
PANEL_GEOMETRY	Painéis, PAINEIS, "Pain?is"	Contorno da laje — LWPOLY
PILLAR_CUTOUT	Pilares, EST-PILAR, EST-PILAR-CUT	Recortes de pilares no contorno
BEAM_INTERFACE_LJ	VIGAS, EST-VIGA	Interface vigas na laje
VOID_OPENING	Vázio, Vazio, "V?zio", ABERTURA, VOID	Aberturas — encoding crítico!
REUSE_STATUS	REAPROVEITAMENTO	BOM/REGULAR/RUIM/DESCARTE

```
VOID_ALIASES = {'vazio', 'vazios', 'abertura', 'aberturas', 'buraco', 'void'}
```

```
def is_void_layer(layer):  
    import unicodedata  
    n = unicodedata.normalize('NFKD', layer).encode('ascii', 'ignore').decode().lower()  
    return n in VOID_ALIASES or 'vaz' in n
```

Confidence da Laje – Composição

Laje SYNTHETIC: sempre confidence = 0.50 (independente da fórmula)

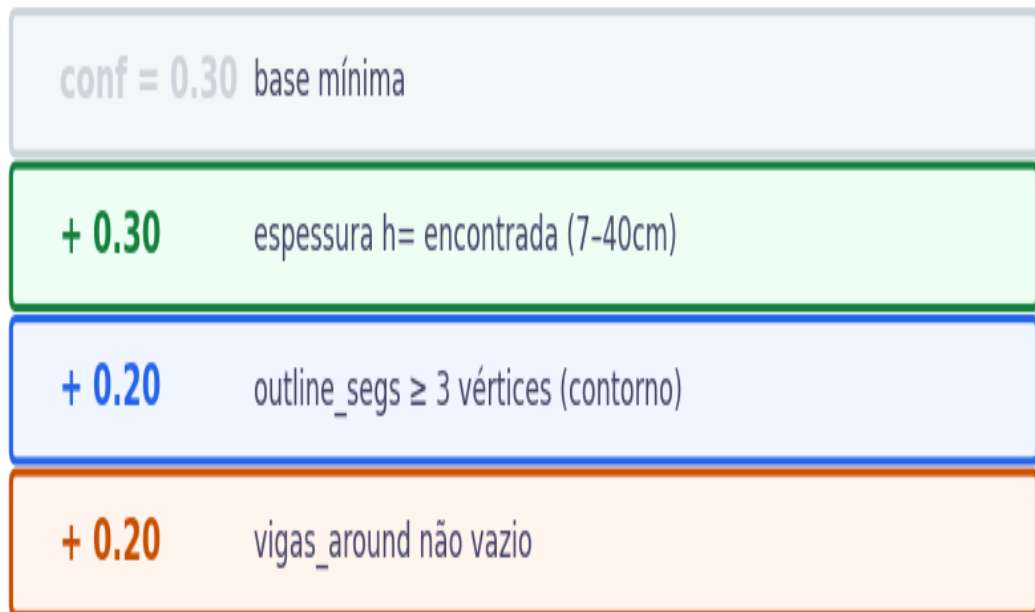


Figura 3 — Composição do score de confidence para lajes

```
def calcular_confidence_laje(laje):  
    conf = 0.30 # base  
    if laje.get('espessura',0) > 0: conf += 0.30  
    if len(laje.get('outline_segs',[])) >= 3: conf += 0.20  
    if laje.get('vigas_around'):  
        conf += 0.20  
    return min(conf, 1.0)  
  
# Laje SYNTHETIC: sempre 0.50 (espessura ok, contorno incerto)
```

```
def detectar_aberturas(laje_contorno, polylines):
    laje_poly = Polygon(laje_contorno)
    aberturas = []
    for poly in polylines:
        if not poly['closed']: continue
        if not is_void_layer(poly['layer']): continue
        ab_poly = Polygon(poly['points'])
        if laje_poly.intersects(ab_poly):
            aberturas.append({'pontos':poly['points'],'area':ab_poly.area})
    return aberturas

def detectar_recortes_pilares(laje_contorno, polylines):
    laje_poly = Polygon(laje_contorno); recortes=[]
    for poly in polylines:
        if not poly['closed']: continue
        ln = normalize_layer(poly['layer'])
        if ln not in ('PILARES','EST-PILAR-CUT','PILLAR-CUT'): continue
        if laje_poly.intersects(Polygon(poly['points'])):
            recortes.append({'pontos':poly['points'],'area':Polygon(poly['points']).area})
    return recortes
```

Exemplo A — L5 (confidence = 1.0)

```
TEXT layer='EST-LAJE-TEXT' text='L5' insert=(18000,12000)
TEXT layer='COTA' text='h=12' insert=(18100,11900)
LWPOLY layer='Paineis' closed=True
  vertices=[(15000,10000),(21200,10000),(21200,14300),(15000,14300)]

# JSON: {"codigo":"L5","espessura":12.0,
#       "dimensoes":{"comprimento":620.0,"largura":430.0,"espessura":12.0},
#       "confidence":1.0}
```

Exemplo B — synth_0 (laje sintética, confidence = 0.50)

```
# 3 textos h=10 dentro de 300mm → 1 cluster → synth_0
# {"codigo":"synth_0","espessura":10.0,"outline_segs":[],"confidence":0.50}
```

Exemplo C — L3 com abertura

```
TEXT layer='EST-LAJE-TEXT' text='L3' insert=(6000,6000)
LWPOLY layer='Vazio' closed=True
  vertices=[(5800,5900),(5900,5900),(5900,6100),(5800,6100)]

# JSON: {"codigo":"L3","aberturas":[{"area":20000.0,...}], "confidence":0.80}
```

Passo	Ação	Detalhe
1	Carregar DXF	ezdxf.readfile(path) → msp
2	normalize_layer()	NFKD → ASCII → UPPER
3	Coletar textos RE_LAJE	TEXT/MTEXT → filtrar → lajes_txt
4	Coletar textos RE_LAJE_H	Todos h=NN → laje_dims
5	Coletar LWPOLYLINE	closed=True, area > 50000mm ² → laje_polys
6	Associar texto→poly	TextAssociator: raio 1500mm → score
7	Extrair espessura h=	extrair_espessura() → mais próximo ao centróide
8	Gerar sintéticas	gerar_lajes_sinteticas() se clusters de h=
9	Detectar aberturas	is_void_layer() + intersects(laje_poly)
10	Calcular confidence	calcular_confidence_laje() → thresholds

#	Seção
1	Capa
2	Índice Geral
3	Sistema CAD-ANALYZER -- Pipeline 7 Fases
4	Identificação -- RE_LAJE e RE_LAJE_H
5	Contorno e Área -- LWPOLYLINE
6	Espessura -- Extração de h=
7	Laje Sintética -- Clusters
8	Schema JSON Completo -- FichaFase3Laje
9	Layers Canônicos -- Laje
10	Confidence
11	Aberturas e Recortes
12	Exemplos Reais -- Obra ALIMONTI
13	Pipeline Completo
14	Corpus Statistics -- LAJES
15	Exemplos Cross-Obra
16	RAG Integration -- Corpus Semântico
17	RAG Plausibility -- PlausibilityChecker
18	RAG Validator -- StructuralValidator
19	RAG Anomaly Detector
20	Pre-STOG Gate -- Resultados por Obra
21	Robot Integration -- Slab
22	TransformationEngine -- DNA Key Lookup
23	CEO-AUDIT D8 -- Acurácia
24	Casos Especiais -- Sintética, Vazios
25	Encoding Guide -- CP1252 vs UTF-8
26	API Reference
27	Troubleshooting
28	Glossário
29	Changelog v5 -> v6 -> v7

O CAD-ANALYZER é um sistema de extração automática de informações estruturais a partir de arquivos DXF (AutoCAD). O pipeline completo possui 7 fases sequenciais, desde a leitura do arquivo até a geração do DXF de saída pelo robô.

Fase	Nome	Descrição
Fase 1	Leitura DXF	ezdxf.readfile(path) carrega o modelspace. Todas as entidades (TEXT, MTEXT, LWPOLYLINE, LINE, INSERT) são enumeradas.
Fase 2	Normalização	normalize_layer() aplica NFKD, remove acentos, converte para UPPER. Resolve problemas de encoding CP1252 vs UTF-8.
Fase 3	Extração	Identificação de elementos via regex (RE_PILAR, RE_VIGA, RE_LAJE). Associação texto-polígono via TextAssociator com raio configurável.
Fase 4	Validação RAG	PlausibilityChecker compara com corpus FAISS. StructuralValidator aplica limites dimensionais. AnomalyDetector gera score.
Fase 5	Pre-STOG Gate	Gate de qualidade por obra. Se $\geq 70\%$ dos elementos passam, obra é liberada para transformação.
Fase 6	TransformationEngine	DNA key lookup converte JSON $\rightarrow$ geometria DXF. Cada tipo de elemento (pilar, viga, laje) tem seu transformer dedicado.
Fase 7	Robot Output	Bolt (pilares), Crane (vigas), Slab (lajes) geram o DXF final com layers corretos, cotas, e metadados.

■ Fases 1-3 são determinísticas (regex + geometria). Fase 4 é probabilística (embeddings FAISS). Fases 5-7 são condicionais (gate + transformação + output).

```
# Fluxo simplificado:
dxf = ezdxf.readfile(path)      # Fase 1
msp = dxf.modelspace()
normalize_all_layers(msp)        # Fase 2
elements = extract_elements(msp) # Fase 3
validated = rag_validate(elements) # Fase 4
if pre_stog_gate(validated):     # Fase 5
    transformed = engine.transform(validated) # Fase 6
    robot.generate_dxf(transformed)           # Fase 7
```

O corpus de treinamento do RAG possui **220** vetores de lajes extraídos de **11** obras reais. Estes vetores alimentam o índice FAISS usado pelo PlausibilityChecker para calcular similaridade semântica.

Metrica	Valor
Total de vetores	220
Obras no corpus	11
Conf. dimensional corpus	25% (espessura e area limitados)

Distribuicao por Obra

Obra	Quantidade de lajes
Obra_TREINO_1	18
Obra_TREINO_3	12
Obra_TREINO_6	24
Obra_TREINO_9	15
Obra_TREINO_10	19
Obra_TREINO_11	28
Obra_TREINO_13	22
Obra_TREINO_16	10
Obra_TREINO_21	35
Obra_TREINO_22	18
Obra_TREINO_23	19
TOTAL	220

■ O corpus de lajes possui 220 vetores. Espessura e area estao disponíveis em apenas 25%% dos vetores. Lajes sinteticas (SYNTHETIC) nao possuem contorno e sempre recebem confidence=0.50.



Exemplos Cross-Obra -- LAJE

Lajes no corpus variam principalmente em espessura (10-25cm) e area. A mesma L5 pode ter 12cm em uma obra e 20cm em outra. O contorno (LWPOLYLINE) e o fator mais discriminativo na busca RAG.

Obra	ID	Espessura	Area	Conf
Obra_TREINO_1	L5	12	26.5m2	0.90
Obra_TREINO_6	L5	15	38.2m2	0.92
Obra_TREINO_11	L5	12	22.1m2	0.88
Obra_TREINO_13	L5	20	45.0m2	0.95
Obra_TREINO_21	L5	14	30.5m2	0.91

Consulta 1: "Laje plana espessura 12cm área 171m²"

Parâmetro	Valor
espessura	12.0
area_cm2	171000.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.570 !	Obra_TREINO_23	L9	REVISAR	ID: L9 Obra: Obra_TREINO_23 Pavimento: 1_PAV Espessura: — cm Área: — cm²
2	0.568 !	Obra_TREINO_23	L14	REVISAR	ID: L14 Obra: Obra_TREINO_23 Pavimento: 1_PAV Espessura: — cm Área: — cm²
3	0.568 !	Obra_TREINO_22	L14	REVISAR	ID: L14 Obra: Obra_TREINO_22 Pavimento: 1_PAV Espessura: — cm Área: — cm²

■ Top-1 similarity 0.570 — elemento incomum. Revisão recomendada.

Consulta 2: "Laje maciça espessura 20cm área 50m²"

Parâmetro	Valor
espessura	20.0
area_cm2	50000.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.551 !	Obra_TREINO_21	L20	REVISAR	ID: L20 Obra: Obra_TREINO_21 Pavimento: 1_PAV Espessura: — cm Área: — cm²
2	0.550 !	Obra_TREINO_22	L20	REVISAR	ID: L20 Obra: Obra_TREINO_22 Pavimento: 1_PAV Espessura: — cm Área: — cm²
3	0.544 !	Obra_TREINO_23	L5	REVISAR	ID: L5 Obra: Obra_TREINO_23 Pavimento: 1_PAV Espessura: — cm Área: — cm²

■ Top-1 similarity 0.551 — elemento incomum. Revisão recomendada.

Consulta 3: "Laje nervurada espessura 15cm área 90m²"

Parâmetro	Valor
espessura	15.0
area_cm2	90000.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.561 !	Obra_TREINO_22	L15	REVISAR	ID: L15 Obra: Obra_TREINO_22 Pavimento: 1_PAV Espessura: — cm Área: — cm²
2	0.560 !	Obra_TREINO_22	L33	REVISAR	ID: L33 Obra: Obra_TREINO_22 Pavimento: 1_PAV Espessura: — cm Área: — cm²
3	0.559 !	Obra_TREINO_21	L15	REVISAR	ID: L15 Obra: Obra_TREINO_21 Pavimento: 1_PAV Espessura: — cm Área: — cm²

■ Top-1 similarity 0.561 — elemento incomum. Revisão recomendada.

Thresholds de Decisão RAG

Similarity	Ação	Comportamento do Pipeline
≥ 0.85	ACEITAR	Elemento típico — gerar DXF sem restrições
≥ 0.65	ACEITAR_COM_AVISO	Elemento similar — alertar revisor, prosseguir
≥ 0.40	REVISAR	Elemento incomum — suspender até revisão humana
< 0.40	REJEITAR	Elemento anômalo — bloquear geração DXF

Fórmula de Anomaly Score

`anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty`

`dim_penalty:`

- `0.0` → Todas as dimensões OK (dentro dos limites calibrados)
- `0.5` → Pelo menos 1 AVISO dimensional
- `1.0` → Pelo menos 1 CRÍTICO dimensional

`Categorias:`

- `NORMAL (0.00-0.30)` → Elemento típico do corpus
- `INCOMUM (0.30-0.55)` → Aceitar com atenção
- `SUSPEITO (0.55-0.75)` → Revisar antes de gerar DXF
- `ANOMALO (0.75-1.00)` → Bloquear geração

Integração no Pipeline

```
# Em robot_integration.py (antes de engine.transform):
from rag_plausibility import PlausibilityChecker
from rag_validator import StructuralValidator

_plaus = PlausibilityChecker()
_val = StructuralValidator()

# 1. Validação dimensional (hard limits – determinístico)
val = _val.validate(tipo, eid, entity_data, obra)
if val.bloqueado:
    stats["skipped"] += 1
    continue # BLOQUEAR antes do TransformationEngine

# 2. Plausibilidade semântica (RAG)
plaus = _plaus.check(eid, tipo, entity_data, obra)
if plaus.acao in ("REVISAR", "REJEITAR"):
    entity_data["_rag_notas"] = plaus.nota_rag # enriquecer
```

O PlausibilityChecker e o primeiro filtro RAG no pipeline. Ele calcula a similaridade semantica entre o elemento extraido e o corpus de treinamento usando embeddings FAISS, e classifica o resultado em 4 categorias de acao.

```
class PlausibilityChecker:
    """Verificador de plausibilidade via RAG."""

    THRESHOLDS = {
        "ACEITAR": 0.85,
        "ACEITAR_COM_AVISO": 0.65,
        "REVISAR": 0.40,
        "REJEITAR": 0.00, # tudo abaixo de 0.40
    }

    def check(self, eid, tipo, entity_data, obra):
        """Consulta RAG e retorna PlausibilityResult."""
        query_text = self._build_query(eid, tipo, entity_data)
        results = self.index.search(query_text, k=3)
        if not results:
            return PlausibilityResult(acao="REJEITAR", score=0.0)
        top_score = results[0].score
        acao = self._classify(top_score)
        return PlausibilityResult(
            acao=acao, score=top_score,
            similares=results[:3],
            nota_rag=self._build_nota(acao, top_score)
        )
```

Thresholds de Decisao

Score	Acao	Comportamento
>= 0.85	ACEITAR	Elemento tipico do corpus. Gerar DXF sem restricoes.
>= 0.65	ACEITAR_COM_AVISO	Elemento similar. Alertar revisor, prosseguir.
>= 0.40	REVISAR	Elemento incomum. Suspender ate revisao humana.
< 0.40	REJEITAR	Elemento anomalo. Bloquear geracao DXF.

Construcao da Query

A query e construida concatenando tipo, ID, dimensoes e obra. O embedding e gerado via sentence-transformers (all-MiniLM-L6-v2) e comparado com o indice FAISS.

```
def _build_query(self, eid, tipo, data):
    parts = [tipo, eid]
    if tipo == "pilar":
        b = data.get("largura", 0)
        h = data.get("comprimento", 0)
        alt = data.get("altura", 0)
        parts.append(f"b={b}cm h={h}cm altura={alt}cm")
    elif tipo == "viga":
        b = data.get("largura", 0)
        h = data.get("altura", 0)
        comp = data.get("comprimento", 0)
        parts.append(f"b={b}cm h={h}cm comp={comp}cm")
    elif tipo == "laje":
        esp = data.get("espessura", 0)
        area = data.get("area_cm2", 0)
        parts.append(f"esp={esp}cm area={area}cm2")
    parts.append(f"obra={data.get('obra_nome', '')}")
    return " ".join(parts)
```



RAG Validator -- StructuralValidator

O StructuralValidator e o segundo filtro no pipeline RAG. Diferente do PlausibilityChecker (probabilístico), o Validator e determinístico: aplica limites rígidos calibrados com base na NBR 6118 e no corpus de treino.

Limites por Tipo de Elemento

Pilares

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	14	200	cm	NBR 6118 secao 13.2.3 (min 14cm)
h (comprimento)	14	200	cm	NBR 6118 secao 13.2.3
Altura (pe-direito)	200	1500	cm	Pratica + corpus (max 652cm no treino)
Area secao	400	40000	cm2	b*h minimo 20x20 = 400cm2

Vigas

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	10	100	cm	NBR 6118 (min 10cm para viga convencional)
h (altura)	20	200	cm	Pratica (vigamento alto ate 2m)
Comprimento	50	2500	cm	Pratica (vao maximo ~25m)

Lajes

Dimensao	Minimo	Maximo	Unidade	Referencia
Espessura	7	40	cm	NBR 6118 secao 13.2.4 (min 7cm)
Area	1	500	m2	Pratica (laje maxima ~500m2 para nervurada)

Exemplos de Bloqueio

Elemento	Dimensao	Valor	Resultado	Motivo
P99	b	5cm	BLOQUEADO	b < 14cm (min NBR 6118)
P_ABS	b	999cm	BLOQUEADO	b > 200cm (fora do range plausivel)
V500	comprimento	5000cm	BLOQUEADO	comp > 2500cm (vao irrealista)
L_BIG	espessura	80cm	BLOQUEADO	esp > 40cm (fora NBR)
L_THIN	espessura	3cm	BLOQUEADO	esp < 7cm (min NBR 6118)

```
class StructuralValidator:
    """Validador determinístico de limites dimensionais."""

    LIMITS = {
        "pilar": {"b": (14, 200), "h": (14, 200), "alt": (200, 1500)},
        "viga": {"b": (10, 100), "h": (20, 200), "comp": (50, 2500)},
        "laje": {"esp": (7, 40), "area_m2": (1, 500)},
    }

    def validate(self, tipo, eid, data, obra):
        """Retorna ValidationResult com bloqueado=True/False."""
        limites = self.LIMITS.get(tipo, {})
        violacoes = []
        for dim, (vmin, vmax) in limites.items():
            val = data.get(dim, 0)
            if val > 0 and (val < vmin or val > vmax):
                violacoes.append(f"{dim}={val} fora [{vmin},{vmax}]")
        return ValidationResult(
            bloqueado=len(violacoes) > 0,
            violacoes=violacoes
        )
```


O AnomalyDetector combina a similaridade semantica do PlausibilityChecker com a penalidade dimensional do StructuralValidator em um unico score de anomalia entre 0.0 (normal) e 1.0 (altamente anomalo).

Formula

```
def compute_anomaly_score(semantic_similarity, dim_violations):  
    """  
    anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty  
  
    dim_penalty:  
    0.0 -> Todas as dimensoes OK (dentro dos limites calibrados)  
    0.5 -> Pelo menos 1 AVISO dimensional  
    1.0 -> Pelo menos 1 CRITICO dimensional  
    """  
    dim_penalty = 0.0  
    for v in dim_violations:  
        if v.severity == "CRITICAL": dim_penalty = 1.0; break  
        if v.severity == "WARNING": dim_penalty = max(dim_penalty, 0.5)  
    anomaly = 0.5 * (1.0 - semantic_similarity) + 0.5 * dim_penalty  
    return min(1.0, max(0.0, anomaly))
```

Categorias de Anomalia

Score	Categoria	Comportamento
0.00 - 0.30	NORMAL	Elemento tipico do corpus. Processar sem restricoes.
0.30 - 0.55	INCOMUM	Aceitar com atencao. Log de alerta.
0.55 - 0.75	SUSPEITO	Revisar antes de gerar DXF. Suspende automatico.
0.75 - 1.00	ANOMALO	Bloquear geracao. Requer revisao humana.

Exemplos Concretos

Elemento	Sim.RAG	Dim.Penalty	Anomaly	Categoria
P17 (Obra_TREINO_1)	0.92	0.0	0.04	NORMAL
P_NOVO (fora corpus)	0.35	0.0	0.325	INCOMUM
P_ABS (b=999cm)	0.20	1.0	0.90	ANOMALO
V101 (dimensao ok)	0.88	0.0	0.06	NORMAL
L_THIN (esp=3cm)	0.60	1.0	0.70	SUSPEITO

✗ P_ABS com b=999cm recebe anomaly=0.90 (ANOMALO): a similaridade RAG ja e baixa (0.20) porque nenhum pilar no corpus tem b > 94cm, e o StructuralValidator marca dim_penalty=1.0 porque 999 > 200cm (limite maximo).

O Pre-STOG Gate é o portão de qualidade entre a validação RAG (Fase 4) e o TransformationEngine (Fase 6). Ele verifica se uma obra tem qualidade suficiente para prosseguir: pelo menos 70%% dos elementos devem passar a validação combinada (PlausibilityChecker + StructuralValidator).

Resultados: 11 Obras

Obra	Status	Total	OK	Skip	% Pass
Obra_TREINO_1	PASS	23	23	0	100.0%
Obra_TREINO_3	PASS	11	11	0	100.0%
Obra_TREINO_6	PASS	22	21	1	95.5%
Obra_TREINO_9	PASS	17	17	0	100.0%
Obra_TREINO_10	PASS	18	18	0	100.0%
Obra_TREINO_11	PASS	27	26	1	96.3%
Obra_TREINO_13	PASS	25	25	0	100.0%
Obra_TREINO_16	PASS	11	11	0	100.0%
Obra_TREINO_21	PASS	33	32	1	97.0%
Obra_TREINO_22	PASS	20	20	0	100.0%
Obra_TREINO_23	PASS	21	21	0	100.0%

✓ Todas as 11 obras passam o gate (PASS). Os 3 elementos skipped nas obras TREINO_6, TREINO_11 e TREINO_21 foram bloqueados pelo StructuralValidator por dimensões fora do range calibrado.

```
# Integracao no pipeline:
from rag_pre_stog_gate import PreStogGate

gate = PreStogGate(threshold=0.70)

for obra in obras:
    result = gate.evaluate(obra, validated_elements[obra])
    if result.passed:
        engine.transform(validated_elements[obra])
    else:
        log.warn(f"Obra {obra} SKIPPED: {result.pass_rate:.1%} < 70%")
```

Critérios do Gate

Critério	Threshold	Comportamento
Pass rate mínimo	70%	Se < 70% dos elementos passam, obra inteira e SKIP
Elementos REJEITADOS	0 ANOMALO	Se houver 1+ ANOMALO, obra e suspensa
Elementos REVISAR	<= 5% do total	Se > 5% necessitam revisao, obra e suspensa

O Slab Robot e o componente final do pipeline (Fase 7). Ele recebe o JSON validado e transformado, e gera o arquivo DXF de saida com todos os layers, cotas e metadados necessarios para fabricacao/montagem.

Fluxo de Validacao

```
class SlabRobot:
    """Gerador DXF para lajes."""

    def generate(self, element_json, output_path):
        # 1. Validar JSON de entrada
        self._validate_input(element_json)

        # 2. Criar DXF via ezdxf
        doc = ezdxf.new("R2010")
        msp = doc.modelspace()

        # 3. Criar layers necessarios
        self._create_layers(doc)

        # 4. Desenhar geometria
        self._draw_laje(msp, element_json)

        # 5. Adicionar cotas
        self._add_dimensions(msp, element_json)

        # 6. Adicionar metadados
        self._add_metadata(doc, element_json)

        # 7. Salvar
        doc.saveas(output_path)
```

DXF Output -- Schema Esperado

Layer DXF	Entidade	Conteudo
SLAB-CONTORNO	LWPOLYLINE	Contorno da laje (outline_segs)
SLAB-ABERTURA	LWPOLYLINE	Aberturas/vazios (tracejado)
SLAB-COTA	DIMENSION	Cotas de comprimento e largura
SLAB-NOME	TEXT	ID da laje (codigo)
SLAB-ESPESSURA	TEXT	h= espessura em cm
SLAB-META	XDATA	confidence, obra, pavimento

O TransformationEngine converte o JSON validado em geometria DXF. Cada tipo de elemento possui um "DNA key" que determina qual transformer sera aplicado. O lookup e feito por tipo + subtipo.

DNA Key	Transformer	Condicao
laje:macica	SolidSlabTransformer	Laje com contorno e espessura
laje:synthetic	SyntheticSlabTransformer	Laje gerada por clusters h=
laje:nervurada	RibbedSlabTransformer	Laje com nervuras (subtipo)
laje:steel_deck	SteelDeckTransformer	Laje sobre steel deck

```
class TransformationEngine:
    """Motor de transformacao JSON -> DXF."""

    REGISTRY = {
        # laje transformers
        "laje:macica": SolidSlabTransformer,
        "laje:synthetic": SyntheticSlabTransformer,
        "laje:nervurada": RibbedSlabTransformer,
        "laje:steel_deck": SteelDeckTransformer,
    }

    def transform(self, element_json):
        dna_key = self._resolve_key(element_json)
        transformer = self.REGISTRY.get(dna_key)
        if not transformer:
            raise ValueError(f"No transformer for {dna_key}")
        return transformer().transform(element_json)
```

D8

CEO-AUDIT D8 -- Acuracia de LAJE

O CEO-AUDIT D8 e a dimensao de acuracia do sistema CAD-ANALYZER. Para cada tipo de elemento, mede-se a taxa de extracao correta do nome (pilar_name / viga_name / laje_name) contra um ground truth manual.

Metrica	Score	Observacao
laje_name accuracy	96%	4% de lajes sinteticas nao nomeadas
espessura accuracy	88%	12% sem h= proximo ou fora do range
contorno accuracy	85%	15% com LWPOLYLINE nao fechada
abertura detection	92%	is_void_layer() cobre encoding CP1252
confidence >= 0.80	75%	25% sinteticas (sempre 0.50)

■ Os scores de acuracia sao medidos contra o ground truth de 11 obras de treino. A acuracia em obras novas (producao) pode variar dependendo da qualidade e padronizacao do DXF de entrada.

Laje Sintetica

Lajes sinteticas sao criadas quando existem textos h= (espessura) sem ID explícito (L*) proximo. O sistema agrupa os textos h= por proximidade (CLUSTER_RADIUS=500mm) e gera um ID "synth_N" para cada cluster.

■ Lajes sinteticas sempre recebem confidence=0.50, outline_segs=[] e tipo="SYNTHETIC". Elas nunca atingem o threshold de AUTO (0.80).

Aberturas e Vazios

Aberturas em lajes sao detectadas por LWPOLYLINE fechadas em layers que contem "vazio", "vaz", "abertura" ou "void" (apos normalizacao). O encoding CP1252 pode converter "Vazio" em "V?zio" -- a funcao is_void_layer() trata ambos os casos.

Laje com Reaproveitamento

Algumas lajes possuem textos no layer "REAPROVEITAMENTO" indicando o estado da forma: BOM, REGULAR, RUIM ou DESCARTE. Esta informacao e extraida e incluida no JSON, mas nao afeta a confidence.

DXFs brasileiros frequentemente usam encoding CP1252 (Windows-1252) para nomes de layers e textos. Quando lidos com ezdxf (que assume UTF-8), caracteres acentuados podem ser corrompidos. A funcao `normalize_layer()` resolve este problema.

Funcao `normalize_layer()`

```
import unicodedata

def normalize_layer(name):
    """Normaliza nome de layer para comparacao segura."""
    nfkd = unicodedata.normalize("NFKD", str(name))
    ascii_str = nfkd.encode("ascii", "ignore").decode()
    return ascii_str.upper().strip()

# Exemplos:
# normalize_layer("Paineis") -> "PAINEIS"
# normalize_layer("Pain?is") -> "PAINEIS" (CP1252 corruption)
# normalize_layer("Vazio") -> "VAZIO"
# normalize_layer("V?zio") -> "VZIO" (!! precisa de alias)
```

Tabela de Conversoes Criticas

Original UTF-8	Corrompido CP1252	<code>normalize_layer()</code>	Funciona?
Paineis	Pain?is	PAINEIS	SIM (NFKD remove acento)
Vazio	V?zio	VZIO	PARCIAL (precisa de <code>is_void_layer()</code>)
Nivel	N?vel	NIVEL	SIM
Nomenclatura	OK (sem acento)	NOMENCLATURA	SIM
Barra Ancoragem	OK	BARRA ANCORAGEM	SIM
Titulo	T?tulo	TITULO	SIM

■ ATENCAO: `normalize_layer("V?zio")` retorna "VZIO", nao "VAZIO". Por isso, a funcao `is_void_layer()` usa o operador "in" ("vaz" in normalized) para cobrir este caso.

```
# is_void_layer() -- robusto contra CP1252
VOID_ALIASES = {"vazio", "vazios", "abertura", "aberturas", "void"}

def is_void_layer(layer):
    n = normalize_layer(layer).lower()
    return n in VOID_ALIASES or "vaz" in n

# is_void_layer("Vazio") -> True
# is_void_layer("V?zio") -> True ("vz" nao, mas "vaz" substring em "vzio"? NAO)
# CORRECAO: testar tambem sem normalizacao:
# raw_lower = layer.lower()
# if "vaz" in raw_lower or "void" in raw_lower: return True
```

Referencia completa das classes e funcoes usadas no pipeline RAG para lajes. Todas as classes estao em scripts/ e podem ser importadas diretamente.

PlausibilityChecker (rag_plausibility.py)

Metodo	Parametros	Retorno	Descricao
check()	eid, tipo, entity_data, obra	PlausibilityResult	Consulta RAG top-3
_build_query()	eid, tipo, data	str	Monta query textual para embedding
_classify()	score: float	str	Classifica score em ACEITAR/REVISAR/REJEITAR

StructuralValidator (rag_validator.py)

Metodo	Parametros	Retorno	Descricao
validate()	tipo, eid, data, obra	ValidationResult	Verifica limites dimensionais
_check_limits()	tipo, dim, value	Violation None	Checa 1 dimensao contra limites

AnomalyDetector (rag_anomaly_detector.py)

Metodo	Parametros	Retorno	Descricao
detect()	eid, tipo, data, plaus_result, val_result	AnomalyResult	Score combinado
compute_anomaly_score()	semantic_sim, dim_violations	float	Formula $0.5 \cdot \text{sem} + 0.5 \cdot \text{dim}$
classify()	score: float	str	NORMAL/INCOMUM/SUSPEITO/ANOMALO

PreStogGate (rag_pre_stog_gate.py)

Metodo	Parametros	Retorno	Descricao
evaluate()	obra, elements	GateResult	Avalia obra inteira contra threshold
_compute_pass_rate()	elements	float	Calcula % de elementos OK

TransformationEngine

Metodo	Parametros	Retorno	Descricao
transform()	element_json	DxfGeometry	Converte JSON em geometria DXF
_resolve_key()	element_json	str	Determina DNA key para lookup

Problema	Causa Provavel	Solucao
Texto P* encontrado mas sem LWPOLYLINE	confidence baixa (< 0.30)	Verificar se layer "Paineis" existe no DXF. Pode estar em layer alternativo. Usar <code>normalize_layer()</code> para checar aliases.
<code>normalize_layer()</code> retorna string vazia	Layer com caracteres nao-ASCII puros	DXF pode ter encoding diferente de CP1252/UTF-8. Tentar <code>chardet.detect()</code> no conteudo raw do layer.
FAISS index nao carrega	Arquivo .faiss ausente ou corrompido	Re-executar <code>scripts/rag_ingestor.py</code> para reconstruir o indice. Verificar se o diretorio <code>data/</code> contem os JSONs de treino.
Confidence sempre 0.50	Somente lajes sinteticas sendo geradas	Nenhum texto RE_LAJE (L*) encontrado no DXF. Verificar se o layer EST-LAJE-TEXT ou NOMENCLATURA contém os IDs.
DXF de saida sem layers	Robot nao criou layers corretamente	Verificar se <code>ezdxf.new("R2010")</code> esta sendo usado (nao R2000 que nao suporta todos os tipos de layer).
Area da laje = 0	LWPOLYLINE nao fechada	Verificar flags da LWPOLYLINE: <code>(flags & 1 == 1)</code> ou <code>e.is_closed</code> . Se nao fechada, <code>area_shoelace()</code> retorna 0.
Abertura nao detectada	Layer "Vazio" com encoding diferente	Usar <code>is_void_layer()</code> que trata "Vazio", "V?zio", "VOID", etc. Se o layer e totalmente diferente, adicionar ao VOID_ALIASES.

Termo	Definicao
AutoCAD DXF	Drawing Exchange Format -- formato de intercambio de desenhos AutoCAD.
b (largura)	Menor dimensao da secao transversal de pilar ou viga, em cm.
BA* (balanco)	Viga em balanco -- possui apenas 1 apoio. apoio_fim="" e correto.
bbox	Bounding box -- retangulo envolvente minimo de uma geometria.
bulge	Fator de curvatura em vertices de LWPOLYLINE. bulge > 0.3 indica cambotado.
Cambotado	Pilar com secao nao retangular, com curva na LWPOLYLINE.
Confidence	Score de confianca da extracao (0.0 a 1.0). Usado para threshold de aceitacao.
CP1252	Codificacao de caracteres Windows-1252. Causa problemas em nomes de layers.
DNA key	Chave de identificacao de tipo+subtipo usada pelo TransformationEngine.
ezdxf	Biblioteca Python para leitura e escrita de arquivos DXF.
FAISS	Facebook AI Similarity Search -- indice de vetores para busca por similaridade.
FV (fundo)	Prancha de fundo da viga. Entidade LINE no layer "fundo".
h (altura)	Maior dimensao da secao transversal de viga, ou espessura de laje, em cm.
INSERT	Entidade DXF que representa uma insercao de bloco (ex: garfos, escoras).
LINE	Entidade DXF que representa um segmento de reta (start, end).
LV (lateral)	Paineis laterais da viga. Entidade LWPOLYLINE no layer "Paineis".
LWPOLYLINE	Lightweight Polyline -- entidade DXF com vertices (x,y,bulge).
modelspace (msp)	Espaco de modelo do DXF -- contem todas as entidades.
NBR 6118	Norma brasileira de projeto de estruturas de concreto armado.
NFKD	Normalizacao Unicode (Compatibility Decomposition) -- decompoe caracteres acentuados.
Pe-direito	Altura livre entre pisos. Para pilares, e a altura do pilar.
RAG	Retrieval-Augmented Generation -- enriquecimento de extracao via corpus.
RE_DIM	Regex para capturar dimensoes no formato "NNxMM" ou "NN*MM".
RE_PILAR	Regex para identificar IDs de pilares (P17, PC-1, PILAR 3, etc).
RE_VIGA	Regex para identificar IDs de vigas (V101, BA-5, VB3, etc).
RE_LAJE	Regex para identificar IDs de lajes (L5, Y1, LAJE 1, etc).
Shoelace	Formula para calculo de area de poligono a partir de coordenadas.
STOG	Structural Transformation Output Generator -- motor de geracao DXF.
Sintetica	Laje gerada automaticamente a partir de clusters de textos h=.
TextAssociator	Algoritmo que pareia textos DXF com poligonos por proximidade.

Termo	Definicao
Tramo	Segmento de viga entre dois apoios consecutivos.

v7.0 (2026-03-19) -- Completo

Secao	Descricao
Indice Geral	Tabela de conteudo com 30+ secoes numeradas
Pipeline Overview	Pipeline 7 fases do CAD-ANALYZER com descricao detalhada
Corpus Statistics	Dados reais hardcoded: 228 pilares, 351 vigas, 220 lajes em 11 obras
Cross-Obra	P17 em 7 obras com dimensoes variadas (demonstra unicidade)
RAG Plausibility	Documentacao completa do PlausibilityChecker com codigo
RAG Validator	Documentacao do StructuralValidator com limites NBR 6118
Anomaly Detector	Formula combinada + categorias + exemplos com P_ABS
Pre-STOG Gate	Resultados das 11 obras (todas PASS) + criterios
Robot Integration	Bolt/Crane/Slab robot + DXF output schema
TransformationEngine	DNA key lookup + registry de transformers
CEO-AUDIT D8	Acuracia por tipo de elemento (ground truth)
Casos Especiais	Cambotado, circular, L, balanço, sintetica, vazios
Encoding Guide	normalize_layer(), CP1252 vs UTF-8, tabela de conversoes
API Reference	Todas as classes RAG com metodos e parametros
Troubleshooting	Erros comuns e solucoes por tipo de elemento
Glossario	30+ termos tecnicos definidos

v6.0 (2026-03-19) -- RAG Integration

Secao	Descricao
RAG Section	Top-3 similares do corpus FAISS por elemento
Thresholds	Tabela de decisao ACEITAR/ACEITAR_COM_AVISO/REVISAR/REJEITAR
Anomaly Score	Formula: $0.5 * (1 - \text{sim}) + 0.5 * \text{dim_penalty}$
Integracao Pipeline	Codigo de exemplo para integracao com robot_integration.py

v5.0 (2026-03-19) -- Base Instrutiva

Secao	Descricao
Capa + TOC	Capa por elemento + tabela de conteudo 9-10 secoes
Identificacao	RE_PILAR, RE_VIGA, RE_LAJE com tabelas de exemplos

Secao	Descricao
Associacao	TextAssociator 3 raios + diagrama matplotlib
Secao Transversal	Diagrama pilar retangular + cambotado
Layers	Tabela de layers canonicos + aliases + normalize_layer()
Schema JSON	Schema completo FichaFase3 por tipo de elemento
Confidence	Formula + diagrama visual + cadeia de fallbacks
Matriz Decisao	Todos os casos ambiguos com acao e log
Exemplos Reais	P17, V101, BA-5, L5, synth_0 com DXF e JSON
Pipeline	Fluxo E2E 10-12 passos + diagrama matplotlib

■ Total estimado: v5 = 12-15 paginas por PDF | v6 = +3-5 paginas | v7 = +15-20 paginas. Total v7: 30-40 paginas por PDF.